

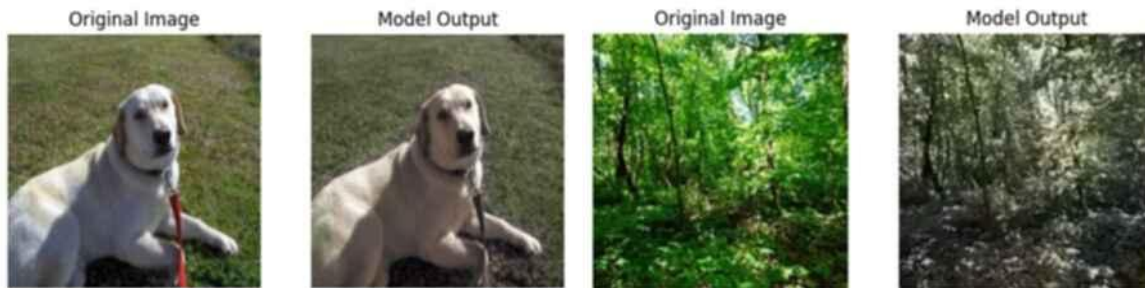
Exploring Image Colorization Techniques using Convolutional Neural Networks

Student Name: Jeffery Downing

Date: December 17th, 2025

Table of Contents

Abstract.....	3
History and Challenges.....	4
Research Question.....	4
Installation	4
Data Configuration	5
Model 1: RGB model for Dogs.....	5
Model 2: LAB model for Dogs	7
Model 3: Complex LAB model	9
Model 4: LAB model for Landscapes	11
Model 5: VGG16 model for Landscapes	12
Conclusion and Future Improvement.....	14
Sources	15
YouTube Videos.....	15



Abstract

This project explores how convolutional neural networks can be used to add color to black-and-white images. Image colorization has applications in restoring historical photographs as well as enhancing photos taken in low-light conditions, such as night-time security footage. Image colorization is a very challenging process that has existed since photography itself. Determining the correct color requires contextual understanding and prior knowledge that may be difficult for a model to pick up. Through a series of experiments, I implemented and evaluated a handful of models using two focused datasets: dogs and landscapes. The results show that colors produced by a model are strongly dependent on the training set. Each model learned specific color patterns and generalized poorly in other domains. For example, the model trained on dogs quickly learned to color backgrounds green, regardless of the actual scene. Similarly, the model trained on landscapes limited itself to blue and yellow tones. Fortunately, incorporating a transfer learning model, specifically VGG16, helped alleviate these problems by enabling it to identify and better handle uncommon scenes. Overall, the process of image colorization remains more of an art than a science. Human input may still be needed, but CNNs can act as a good starting point for generating colored images.

- Short YouTube video: <https://youtu.be/FuStEo3ykBc>
- Long YouTube video: <https://youtu.be/4kajcQwvSkM>

History and Challenges

Image colorization dates back to the earliest days of photography in the late 1800s, when artists manually painted color onto images or used processes like photochrom to create tinted images. In the era of film, studios experimented with chemical tinting and toning to add colors. In the current age of digital media, most colorization is done with software. While the process of adding color to an image has become easier, ensuring accuracy still requires contextual knowledge and research. For example, understanding the traditional colors used by a tribe is necessary to accurately colorize their clothing. Similarly, an artist unfamiliar with a company's historical branding, such as 7up's old red-and-black logo, could unknowingly colorize the image using the company's modern colors. [1]

Recognizing these challenges can help us understand how a CNN may struggle. A CNN relies entirely on its training set when predicting colors. Since old photographs do not have colored versions to train on, our models may be biased towards patterns in the modern age. Such a model would be unable to accurately depict the patterns seen in the past. Moreover, a CNN cannot research or use external knowledge to guide its decisions. Because of this, its predictions will exclusively reflect the patterns it has learned. Knowing this, we should be careful about taking the model's output as is.

Research Question

This project is interested in asking how model architecture and dataset bias impact the performance and generalization of CNN-based image colorization.

Installation

The following python libraries will need to be installed to run the code:

- opencv-python
- numpy
- matplotlib
- tensorflow

Data Configuration

Two datasets were used for this project: the dogs_vs_cats dataset from week five of class and a landscapes dataset from Kaggle. Included in the zip file are 80 training images, 10 validation images, and 4 to 5 testing images for each. The main code is in **Image_Colorization_Code.ipynb** and can be run right away. However, I trained my models using 1000 training images, and if you wish to use the same dataset, then please follow these steps:

- Create a folder called 'data' in the same directory as the notebook files.
- Download the dog_vs_cats.zip file from Canvas.
 - https://canvas.harvard.edu/courses/164142/pages/week-5-2?module_item_id=1976712
- In the dogs_vs_cats.zip file, extract train.zip
- In the train.zip file, extract the train folder and put it in the 'data' folder.
- Download the landscapes dataset from Kaggle.
 - <https://www.kaggle.com/datasets/theblackmamba31/landscape-image-colorization/data>
- Extract the 'landscape Images' folder and put it in the 'data' folder.
- Run all the cells in the **Image_Colorization_Data_Setup.ipynb**.
 - This will create two new folders ('dog_images' and 'landscape_images') in the main directory. These contain the training and validation subsets.

Model 1: RGB model for Dogs

For the first model of this project, I decided to train a model using images of dogs from the dogs_vs_cats dataset provided in week 5 of class. I used a subset of 1000 images for training, and 500 for validation. A grayscale image was used as the input, and the RGB color channels were used as the output. To approach the problem, I created an overcomplete autoencoder. Each layer of the encoder used 3x3 convolutions while also reducing the image size with a 2x2 max-pooling layer. The encoder started with 32 nodes, and each layer doubled the amount until the bottleneck layer of 256 nodes. The decoder was implemented similarly, but used 2x2 up-sampling layers to return the image to its original size. A final convolutional layer

was added to predict the three RGB color channels. Unfortunately, the results were non-existent. The model failed to reconstruct the image and only produced a brown box. It's clear that too much was being asked of the model. It had to reconstruct the entire image and add color. I decided to explore other methods.

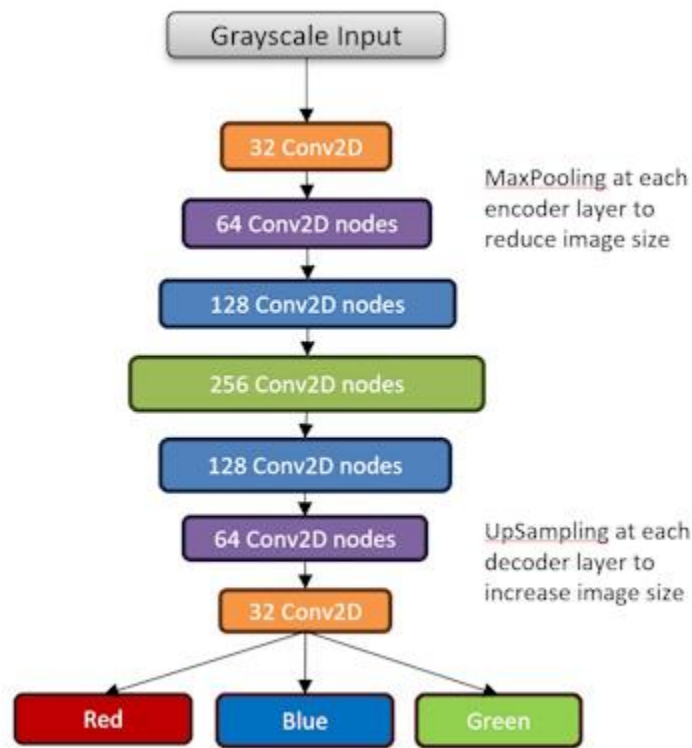


Figure 1: Architecture of Model 1



Figure 2: Test example from Model 1

Model 2: LAB model for Dogs

After the previous model failed to reconstruct images, I reconsidered my approach. I realized that the model shouldn't need to reconstruct all three color channels. Since we have the grayscale image, we should be able to calculate the third color using the other two. Upon further research, I learned about the LAB color format, which uses a light channel (L) and two color channels (A and B), corresponding to the red-green and yellow-blue axes [2]. I reused the same overcomplete autoencoder from Model 1, but modified the final layer to output two AB channels instead of three RGB channels.

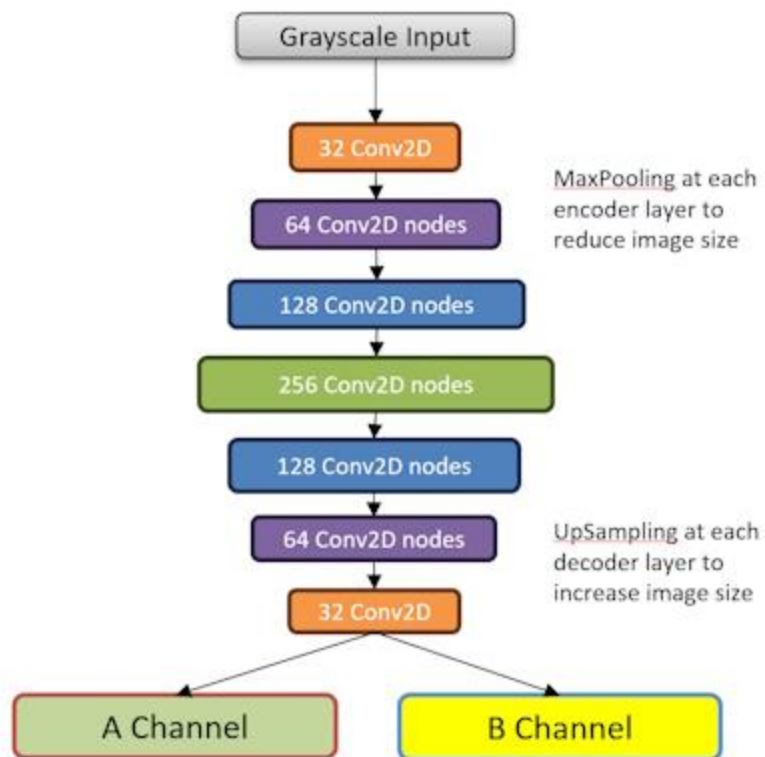
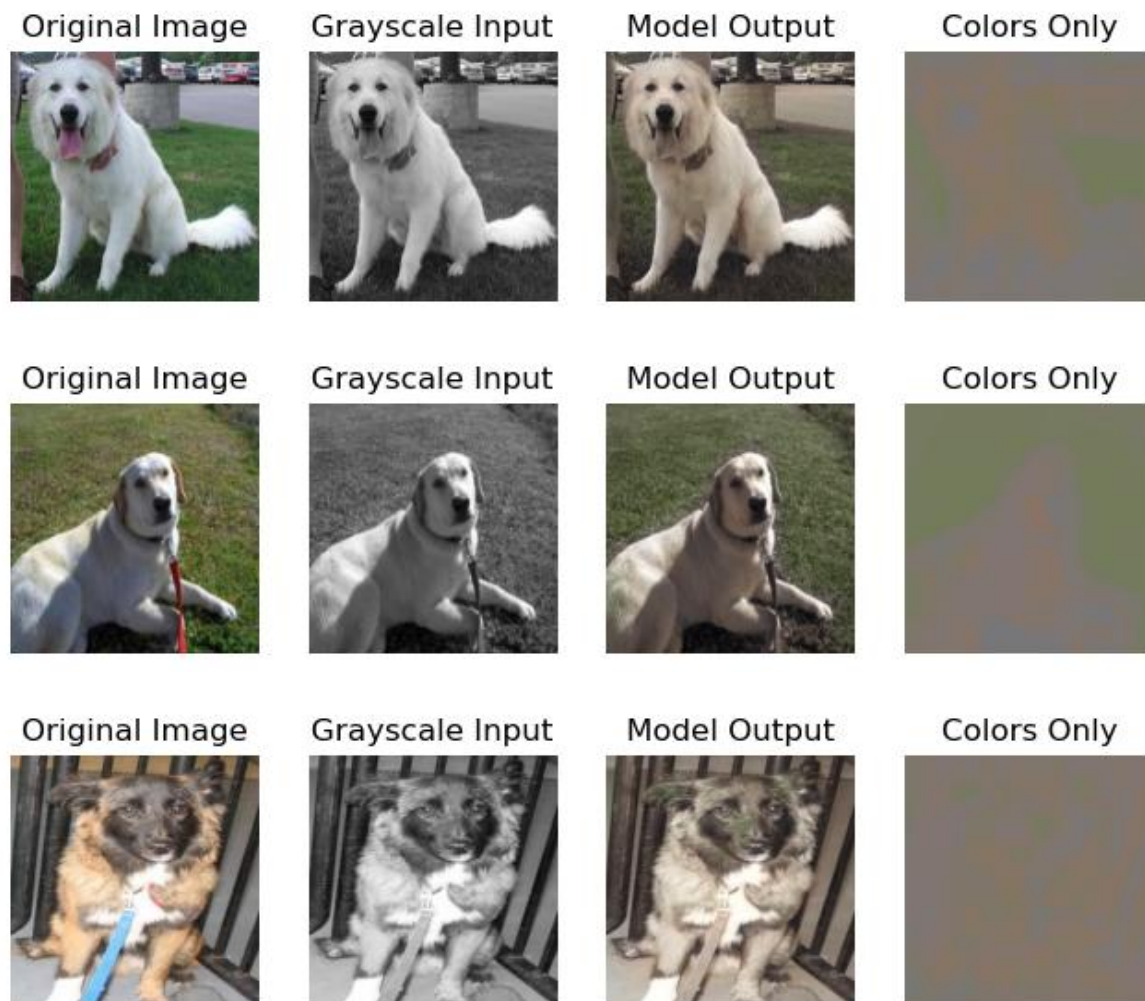


Figure 3: Architecture of Model 2

Image Colorization using CNNs

Jeffery Downing

This model performed much better and produced decent results. The model assigned a reasonable color to each dog and would fill the background with green, regardless of what was there. It appears the model learned that a fair number of images had grass in the background. This provided decent outputs for the first two images, although it colored the first dog gold instead of white. However, it was unable to color other scenes and sometimes applied patches of green on the dog itself. In the last image, we see that the model colored the white concrete green. The model appears to be heavily influenced by its training data.



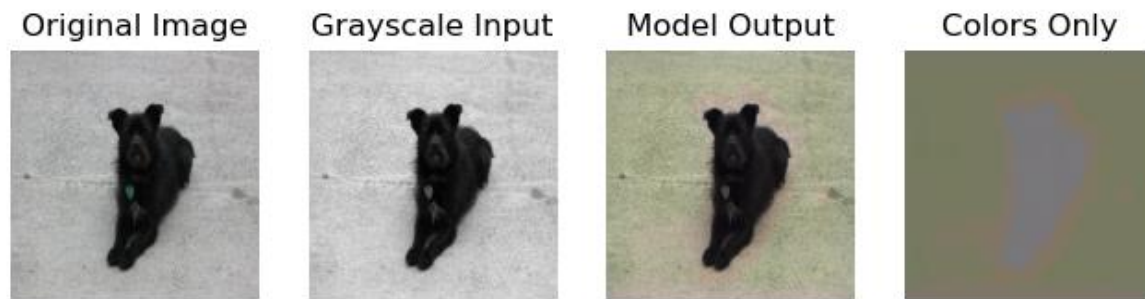


Figure 4: Results of Model 2

Model 3: Complex LAB model

For my third model, I decided to see if a more complex network could improve performance. I incorporated the skip connections used in the U-Net architecture [3]. The outputs of each block in the encoder are concatenated with the corresponding decoder block, which allows the model to reuse any information that may have been lost during down-sampling. Each block contained two convolutional layers along with batch normalization. Despite these efforts, the model performed worse than Model 2. Given this, I chose to abandon this architecture and continue working with the simpler autoencoder.

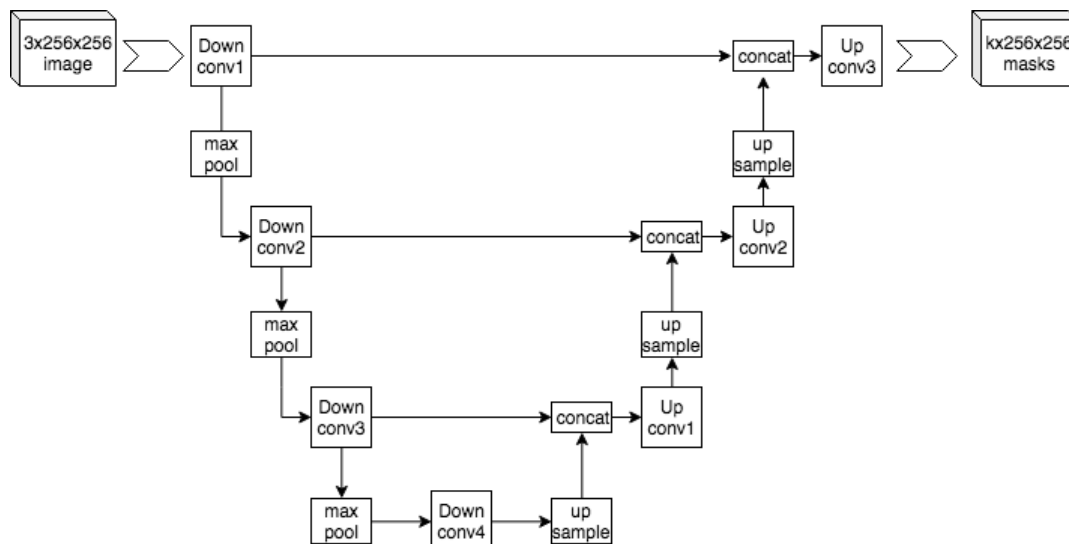


Figure 5: U-Net Architecture used for Model 3 [4]

Image Colorization using CNNs
Jeffery Downing

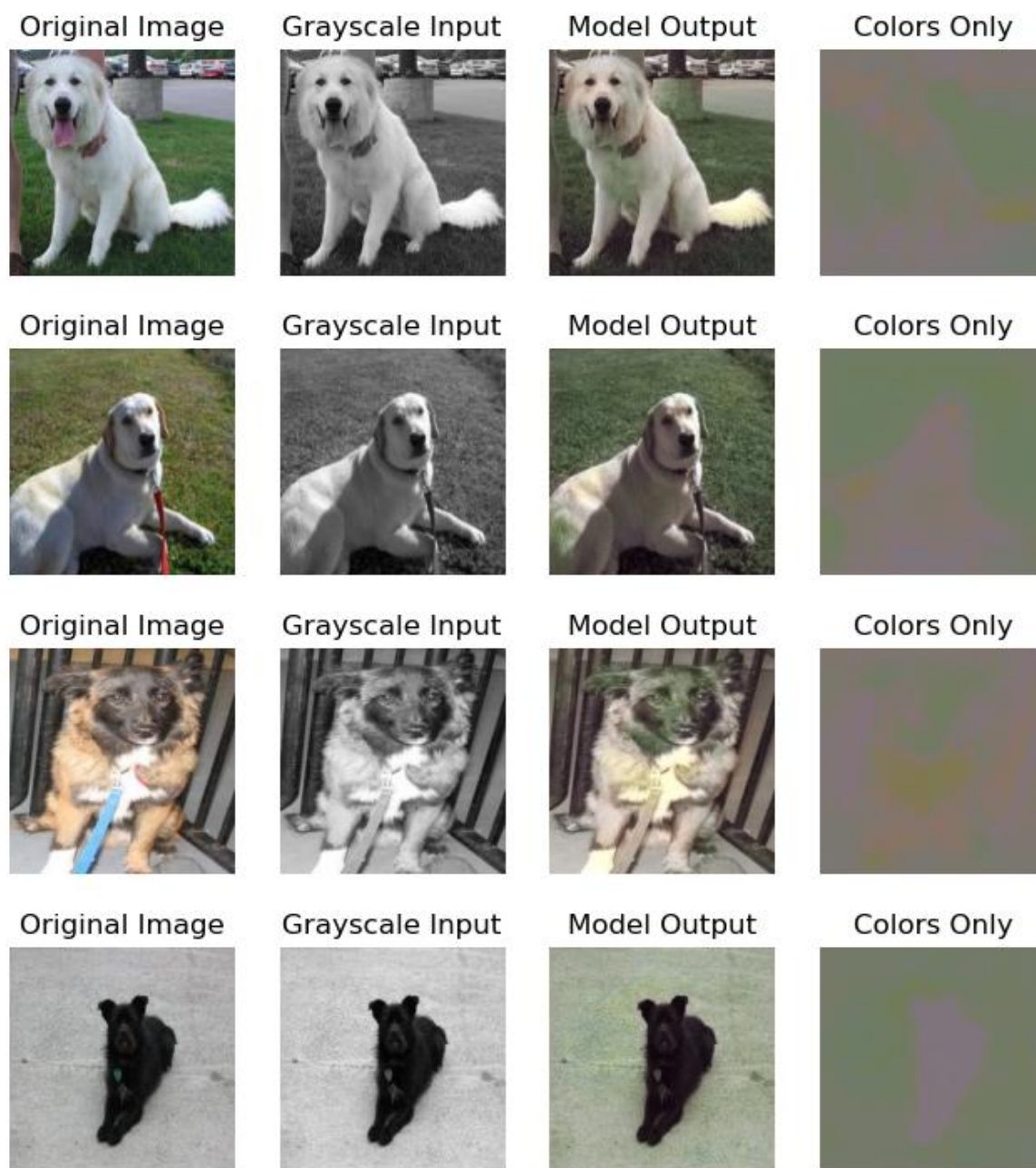


Figure 6: Results of Model 3

Model 4: LAB model for Landscapes

It was clear that the previous models relied heavily on their training set of dog images, so I wanted to see how a different dataset would influence the model's behavior. This time, I used images of landscapes with a wide range of scenes, including beaches, oceans, forests, and urban buildings. My goal was to see if the model's architecture could learn to identify and reproduce the colors for each type of environment. Unfortunately, the model struggled to learn and was unable to map different colors to each scene. There were too many competing visual patterns, ranging from blue oceans, green forests, white snow, and colorful buildings. The model defaulted to blue and yellow tones and was unable to accurately fill in the colors. I decided to explore ways to resolve this problem.

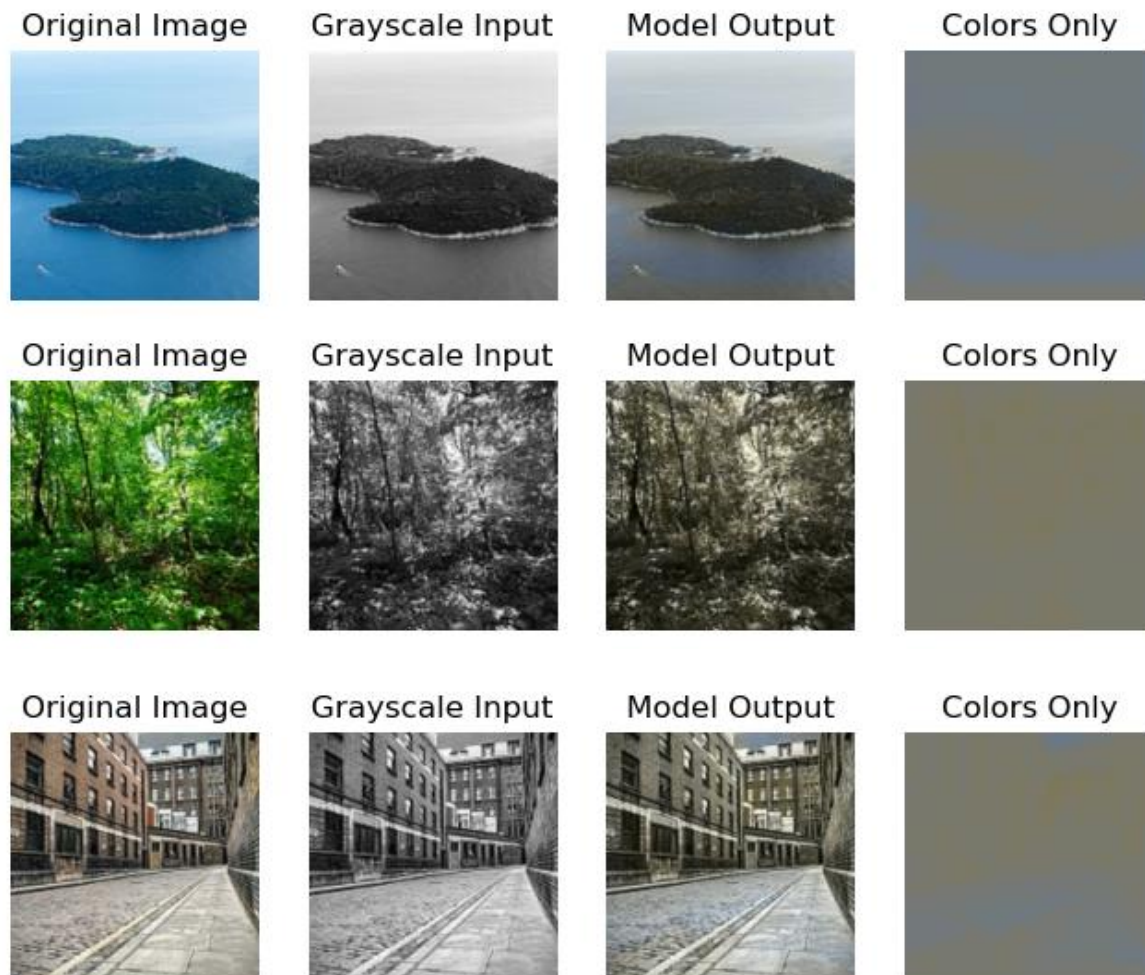




Figure 7: Results of Model 4

Model 5: VGG16 model for Landscapes

For my final model, I implemented a transfer learning approach using the VGG16 architecture for the encoder. VGG16 requires three color channels as input, so I copied the grayscale layer into all three channels. I extracted the "block5_conv3" layer and ran it through the same decoder as my other models. All VGG16 layers were frozen, so only the decoder was trained.

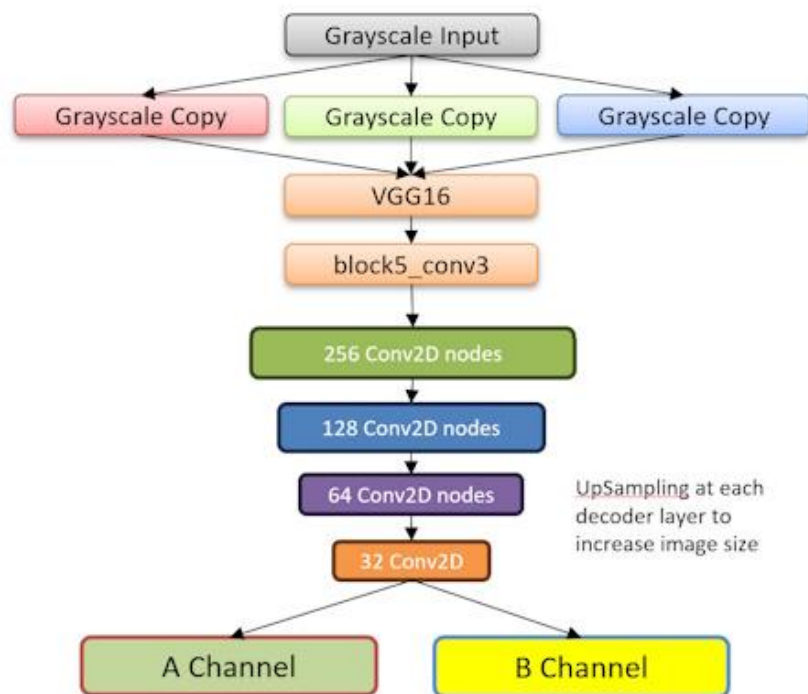


Figure 8: Architecture for Model 5

Image Colorization using CNNs

Jeffery Downing

This significantly improved the colorization, as the model was now able to use more colors than just blue and yellow. While the colorization quality still wasn't perfect, the model was now able to color forests with green and brick buildings with red. Using transfer learning enabled the model to identify more features and learn more diverse color patterns.

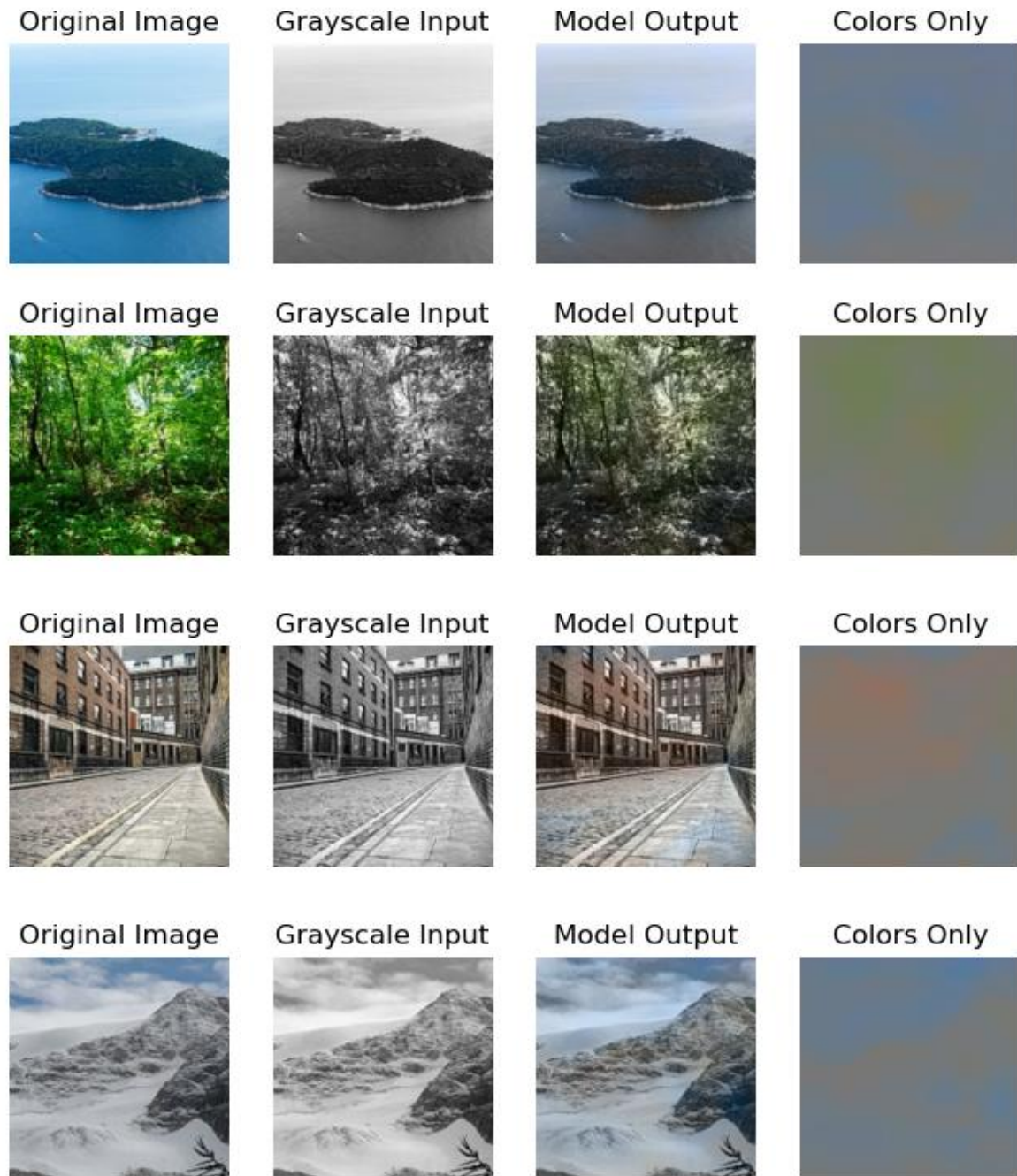


Figure 9: Results for Model 5

Conclusion and Future Improvement

This project aimed to discover the strengths and weaknesses of using convolutional neural networks in image colorization. Utilizing LAB color channels allowed the model to focus purely on creating colors instead of forcing it to also reconstruct entire images. Through my investigation, I found that overcomplete autoencoders work well with identifying color patterns, but increasing the architecture's complexity does not necessarily lead to improved results. The patterns learned are highly dependent on the training set, so using a model to color anything outside that domain will not produce accurate results. Transfer learning offered promising results and can help when the dataset contains different kinds of images by leveraging its pretrained features.

For future improvements, I would start by expanding the size and diversity of the training dataset. The specialized datasets used in this project limited the model's exposure to certain scenes and color patterns, which contributed to bias and poor generalization. Additionally, I would experiment with tuning the pretrained VGG16 layers to see if performance can be improved. Another improvement would be to incorporate some form of human guidance. For example, allowing the user to specify the color of a particular object, or identifying what an object in the image represents to help it make better colorizations. Even with all these advancements, image colorization remains a difficult task. Human input may still be needed, but CNNs can act as a good starting point for generating colored images.

Sources

[1] Vox. How obsessive artists colorize old photos <https://youtu.be/vubuBrcAwY>

[2] NumberAnalytics. The Ultimate LAB Color Guide
<https://www.numberanalytics.com/blog/ultimate-lab-color-guide>

[3] medium.com. The U-Net: A Complete Guide
<https://medium.com/@alejandro.itoaramendia/decoding-the-u-net-a-complete-guide-810b1c6d56d8>

[4] Wikipedia. U-Net: <https://en.wikipedia.org/wiki/U-Net>

Dog_vs_cats dataset: https://canvas.harvard.edu/courses/164142/pages/week-5-2?module_item_id=1976712

Landscape dataset:
<https://www.kaggle.com/datasets/theblackmamba31/landscape-image-colorization/data>

YouTube Videos

- Short YouTube video: <https://youtu.be/FuStEo3yBc>
- Long YouTube video: <https://youtu.be/4kajcQwvSkM>